

Data Analysis using Python

04 – Time Series

Contents

Timeseries basics

Slicing

Ranges and frequencies

Resampling

Shifting

Moving windows - rolling and expanding

Aggregating data

Timeseries Basics

What is a Time Series?

A time series is a sequence of data points collected or recorded at successive points in time.

Common examples:

Stock prices, temperature readings, sales data, etc.

Why Use Pandas for Time Series?

Pandas provides powerful tools to:

Parse, manipulate, and analyze time series data.

Handle datetime objects seamlessly.

Resample, aggregate, and visualize data efficiently.

Timeseries Basics

DatetimeIndex:

pandas leverages **DatetimeIndex** for indexing and slicing time series.

Automatic Parsing:

pandas can convert strings to datetime objects.

Operations:

Easily perform calculations like resampling, shifting, and rolling operations.

Timeseries Basics

```
import pandas as pd

# Sample time series data
data = {'value': [10, 15, 13, 18, 20]}
index = pd.date_range(start='2024-01-01', periods=5, freq='D')

# Create a time series DataFrame
ts = pd.DataFrame(data, index=index)

# Display the time series
print(ts)

# Accessing data for a specific date
print(ts.loc['2024-01-02'])

# Plot the time series
ts.plot(title='Simple Time Series', ylabel='Value')
```

Slicing

Why Slice Time Series Data?

Allows extracting subsets of data for specific time ranges.

Useful for analyzing data within a specific timeframe (e.g., daily, monthly, yearly).

Key Features of Slicing in Pandas:

DatetimeIndex Support:

Use `.loc[]` with strings representing dates or ranges.

Partial String Indexing:

Automatically selects all data for the specified year, month, or day.

Flexible Ranges:

Specify start and end dates for slicing.

Slicing

```
import pandas as pd

# Create sample time series data
data = {'value': [10, 15, 13, 18, 20, 25, 30]}
index = pd.date_range(start='2024-01-01', periods=7, freq='D')
ts = pd.DataFrame(data, index=index)

# Slicing for a specific day, date range and month (partial string indexing)
daily_data = ts.loc['2024-01-02']

range_data = ts.loc['2024-01-02':'2024-01-04']

monthly_data = ts.loc['2024-01']

# Display the results
print("Daily Data:\n", daily_data)
print("\nRange Data:\n", range_data)
print("\nMonthly Data:\n", monthly_data)
```

Ranges and Frequencies

What are Ranges and Frequencies?

Ranges: Specify the start, end, and intervals of time series data.

Frequencies: Define the time interval (daily, monthly, hourly, etc.) for a time index.

Key Functions:

`pd.date_range:`

Generates a range of dates with specified start, end, and frequency.

`reindex:`

Adjusts or aligns an existing time series to a new time index, filling missing values.

Ranges and Frequencies

Using `pd.date_range` to Create Time Ranges

```
import pandas as pd

# Generate a daily time range
daily_range = pd.date_range(start='2024-01-01', end='2024-01-07', freq='D')

# Generate an hourly time range
hourly_range = pd.date_range(start='2024-01-01', periods=24, freq='h')

print("Daily Range:\n", daily_range)
print("\nHourly Range:\n", hourly_range)
```

Using `reindex` to Align Data

```
# Create a sample time series
data = {'value': [10, 15, 20]}
index = pd.date_range(start='2024-01-01', periods=3, freq='D')
ts = pd.DataFrame(data, index=index)

# New index with a different range
new_index = pd.date_range(start='2024-01-01', end='2024-01-05', freq='D')

# Reindex the time series
aligned_ts = ts.reindex(new_index, fill_value=0)

print("\nOriginal Time Series:\n", ts)
print("\nReindexed Time Series:\n", aligned_ts)
```

Resampling

What is Resampling?

Resampling adjusts the frequency of time series data by **up-sampling** or **down-sampling**.

Up-Sampling: Increase the frequency (e.g., daily to hourly).

Down-Sampling: Decrease the frequency (e.g., daily to weekly).

Key Concepts:

resample:

Groups data into new time intervals based on specified frequency.

Filling Missing Data:

Handle gaps in data after up-sampling using **fill** or **interpolation**.

Resampling

Down-Sampling:

Aggregate data to a lower frequency.

```
import pandas as pd

# Sample time series
data = {'value': [10, 15, 13, 18, 20]}
index = pd.date_range(start='2024-01-01', periods=5, freq='D')
ts = pd.DataFrame(data, index=index)

# Down-sample to weekly frequency and calculate mean
weekly = ts.resample('W').mean()

print("Weekly Resampled Data:\n", weekly)
```

Resampling

Up-Sampling:

Increase the frequency and handle missing data.

```
# Up-sample to hourly frequency with forward fill
hourly = ts.resample('h').ffill()

print("\nHourly Resampled Data (Forward Fill):\n", hourly)
```

Filling and Interpolation:

Fill gaps using interpolation.

```
# Up-sample to hourly frequency with interpolation
hourly_interpolated = ts.resample('h').interpolate(method='linear')

print("\nHourly Resampled Data (Interpolation):\n", hourly_interpolated)
```

Shifting

What is Shifting?

Shifting moves data forward or backward in time without changing the index.
Useful for calculating differences or lags in time series data.

Key Concepts:

shift

Moves data by a specified number of time periods.
Positive values shift forward; negative values shift backward.

Applications:

Calculate day-on-day changes, rolling returns, or lagged values

Shifting

```
import pandas as pd

# Sample time series data
data = {'value': [100, 120, 115, 130, 140]}
index = pd.date_range(start='2024-01-01', periods=5, freq='D')
ts = pd.DataFrame(data, index=index)

# Calculate the previous day's value
ts['previous_day'] = ts['value'].shift(1)

# Calculate day-on-day change
ts['day_on_day_change'] = ts['value'] - ts['previous_day']

# Display the result
print(ts)
```

Moving Windows

What are Moving Windows?

Moving windows calculate statistics (e.g., mean, sum) over a rolling or expanding window of time.

Two main types:

Rolling Window:

Fixed-size window that moves through the data.

Expanding Window:

Window grows to include all data up to the current time.

Key Concepts:

rolling:

performs calculations on a sliding window of a specified size.

expanding:

calculates cumulative statistics over all prior data.

Moving Windows

```
import pandas as pd

# Sample time series data
data = {'value': [100, 120, 115, 130, 140, 150, 160]}
index = pd.date_range(start='2024-01-01', periods=7, freq='D')
ts = pd.DataFrame(data, index=index)

# Calculate a 3-day rolling mean
ts['rolling_mean'] = ts['value'].rolling(window=3).mean()

# Calculate an expanding mean
ts['expanding_mean'] = ts['value'].expanding().mean()

# Display the result
print(ts)
```

Aggregating Data

What is Aggregation in Time Series?

Aggregation summarizes data over a specified time period.

Commonly used with the resample method to adjust data frequency and compute statistics (e.g., sum, mean).

Key Concepts:

resample:

Groups data into specified time intervals.

Aggregation Functions:

Compute statistics such as **mean**, **sum**, **min**, **max**, etc.

Applications:

Aggregate daily data to weekly, monthly, or yearly summaries.

Aggregating Data

```
import pandas as pd

# Sample time series data
data = {'value': [100, 120, 115, 130, 140, 150, 160]}
index = pd.date_range(start='2024-01-01', periods=7, freq='D')
ts = pd.DataFrame(data, index=index)

# Resample to weekly frequency and calculate statistics
weekly_sum = ts.resample('W').sum() # Weekly total
weekly_mean = ts.resample('W').mean() # Weekly average

# Display the aggregated data
print("Weekly Sum:\n", weekly_sum)
print("\nWeekly Mean:\n", weekly_mean)
```

